# Ch4 選擇結構

# Ch4 選擇結構

- 程式的三個主要結構為循序結構、選擇結構與重複結構。
  - 1.循序結構：
  - 為程式有從第一行開始逐行執行的特性，第一行執行完畢後執行第二行，第二行執行完畢後執行第三行，直到程式執行結束

- 2.選擇結構：
- 為若條件測試的結果為真，則做條件測試為真的動作，否則執行條件測試為假的動作
- 例如：若成績大於等於60 分，則輸出及格，否則輸出不及格。

- 3.重複結構：
- 讓電腦重複執行某個區塊的程式多次，電腦適合做重複的工作
- 例如：求1+2+3+…+1000，使用重複結構可在很短時間內重複執行相加的程式，直到求出加總的結果，重複結構將於下一章介紹。善用這三種結構可以寫出解決問題的程式。

# 流程圖圖示介紹

表 4-2　流程圖圖示介紹

| 流程圖圖示 | 意義 |
| --- | --- |
|  | 程式的流程，表示程式的處理順序，表示循序結構。 |
|  | 條件選擇，於菱形內寫入條件判斷，表示選擇結構。 |
|  | 程式的敘述區塊，寫出所需完成的功能。 |
|  | 程式的開始或結束。 |
|  | 程式所需的輸入與輸出。 |
|  | 流程圖的連接點，代表流程圖的進入點或離開點。當流程圖過長可以使用連接點將一個過長流程圖切割成多個流程圖組合起來。 |

# Ch4 選擇結構

- 日常生活中也有許多選擇結構的對話，「若明天天氣好的話，我們就去游泳，否則就待在家裡」
- 程式語言提供選擇結構的程式結構，讓使用者可以於程式中使用，邏輯上的語意為「若測試條件成立，則執行條件成立的動作，否則執行條件不成立的動作」

- 許多問題的解決過程，都會遇到選擇結構，如登入系統時需要驗證帳號和密碼，正確則可登入系統，否則跳到登入畫面，重新輸入帳號與密碼。

# Ch4 選擇結構

- 選擇結構分成
  - 單向選擇結構
  - 雙向選擇結構
  - 多向選擇結構
  - 巢狀選擇結構

# 4-1 單向選擇結構

- 單向選擇結構是最簡單的選擇結構，日常生活上經常用到，例如：「若週末天氣好的話，我們就去打球」。
- 單向選擇結構只做測試條件為真時，執行條件為真的動作，只有一個方向的選擇，因此稱做單向選擇結構。

# 選擇結構：單向選擇結構流程圖

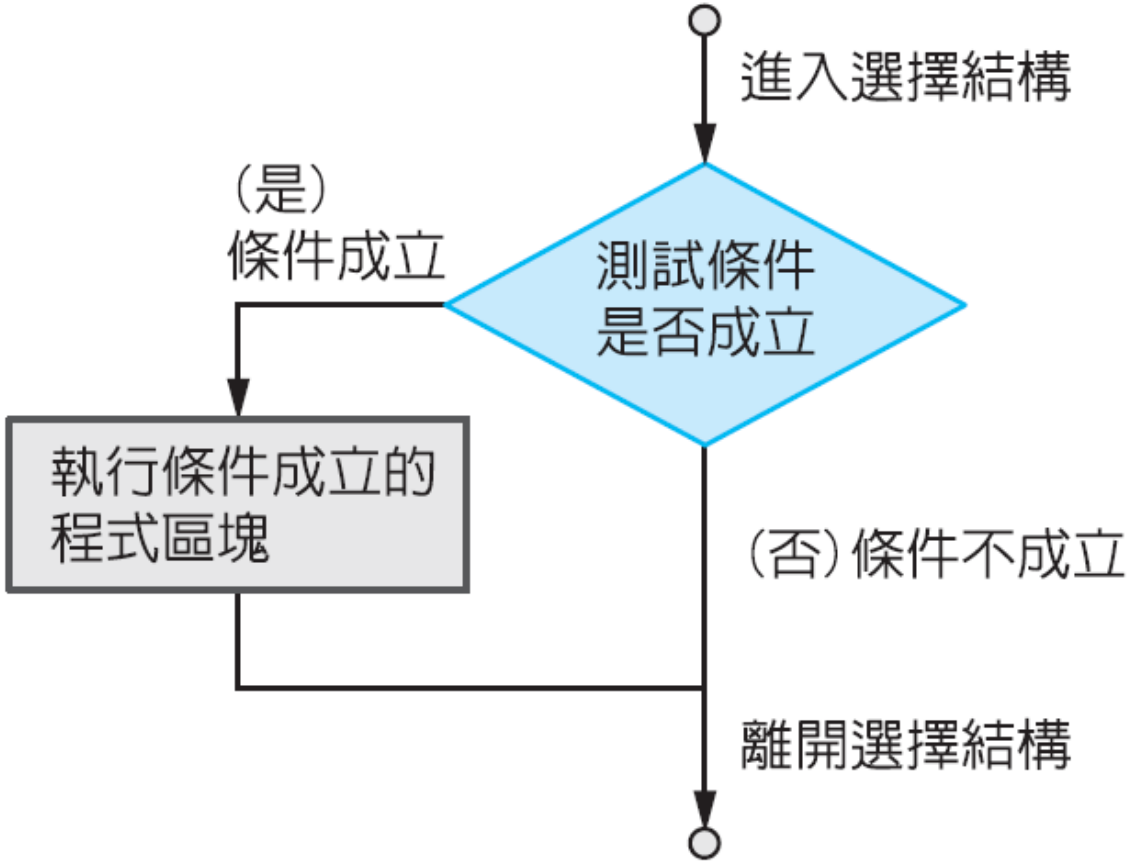



圖 4-1　單向選擇結構流程圖

# 4-1 單向選擇結構

表 4-1 單向選擇結構

| 單向選擇程式語法 | 程式範例 |
| --- | --- |
| if 條件判斷 :<br>　條件成立的敘述 | if score >= 60:<br>　print(" 很好，請繼續保持下去 ") |
| 說明 | |
| 若變數 score 大於等於 60，則顯示「很好，請繼續保持下去」。 | |

# 4-1　單向選擇結構

- Python 程式撰寫的注意事項，有以下幾點：
  - 1.條件成立要做的動作有哪些，可能不只一行程式，那到底有幾行，Python 程式碼以**縮行**表示執行的範圍。

- 2.條件成立要做什麼，需要執行的每一行程式碼都要縮相同的空白鍵(space) 個數，通常使用4 個空白鍵
- 所以「if 條件判斷 :」後面接的條件成立的動作，每一行都需要以4 個空白鍵開頭，表示執行的範圍。

– 3.Tab 也可用於表示縮行，但是空白鍵與Tab鍵不要混用，整個程式只能從頭到尾選擇其中一種進行縮行。

# 範例4-1-1 判斷及格 ( ch4\4-1-1- 判斷及格.py)

- 寫一個程式判斷所輸入成績是否及格，成績及格則顯示「很好，請繼續保持下去」。
- 解題想法
  - 可以使用單向選擇結構撰寫程式，判斷成績是否及格，及格就顯示「很好，請繼續保持下去」。

# 範例4-1-1 判斷及格 流程圖

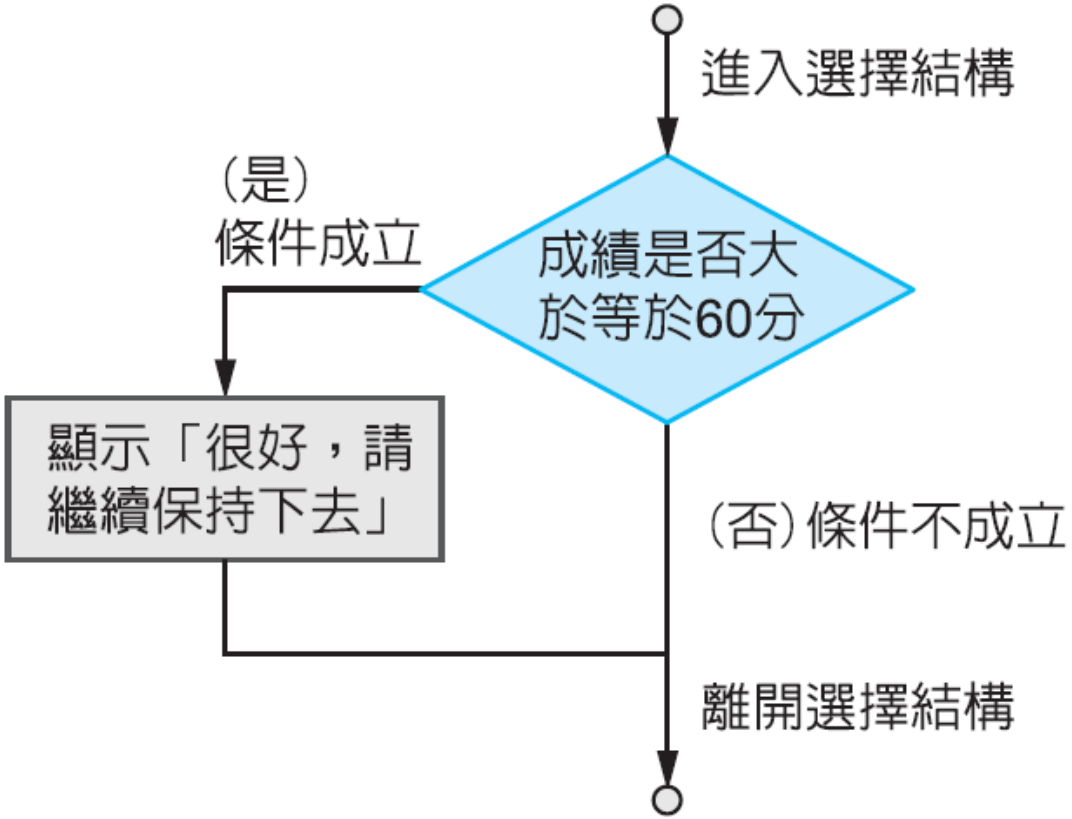



圖 4-2　判斷及格流程圖

# 範例4-1-1 判斷及格 ( ch4\4-1-1- 判斷及格.py)

- 程式碼

```
01 score = int(input(' 請輸入一個成績？ '))
02 if score >= 60:
03     print(' 很好，請繼續保持下去')
```

- 程式解說
  - **第1 行：**於螢幕顯示「請輸入一個成績？」，允許使用者輸入成績，並經由int 函式轉換成整數物件，變數score 參考到此整數物件。
  - **第2 到3 行：**使用條件判斷(if) 對score 做判斷，若大於等於60 分，就輸出「很好，請繼續保持下去」。

## 執行結果

輸入成績「60」，結果顯示在螢幕。

```
請輸入一個成績？ 60
很好，請繼續保持下去
```

# 4-2 雙向選擇結構

- 雙向選擇結構比起單向選擇結構更複雜一些，日常生活上屬於雙向選擇的對話，例如：「若週末天氣好的話，我們就出去打球，否則去看電影」。
- 雙向選擇結構為當測試條件為真時，執行測試條件為真的動作；否則做測試條件為假的動作。有兩個方向的選擇，因此稱做雙向選擇結構。

# 選擇結構：雙向選擇結構 流程圖

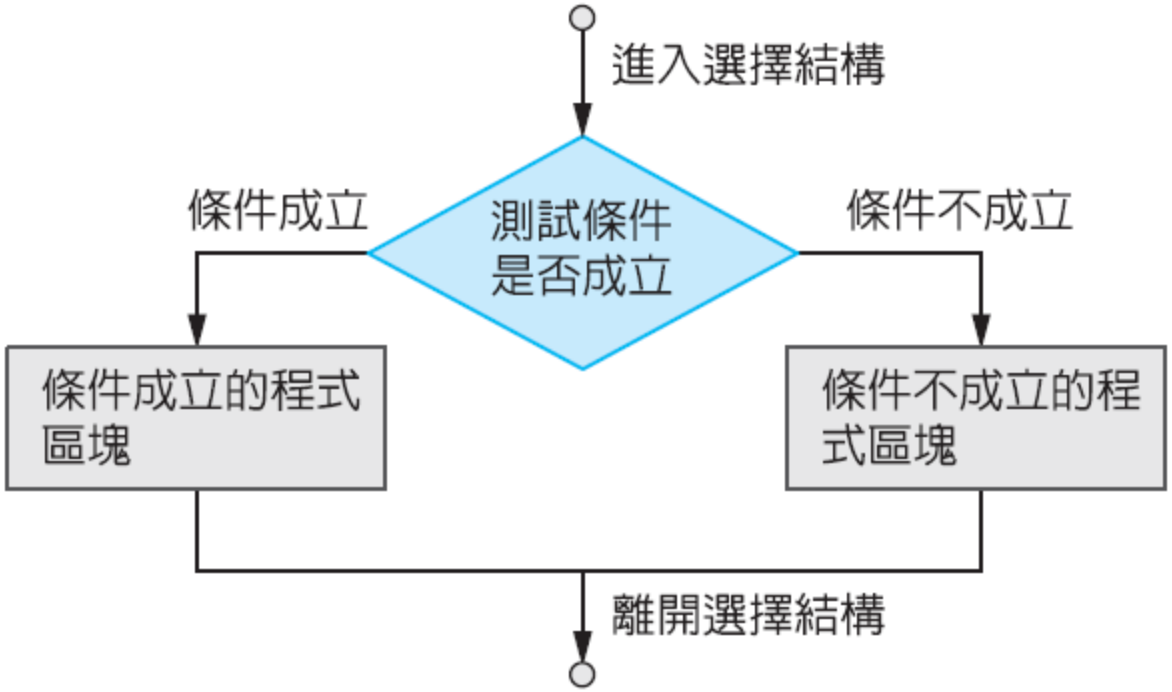



圖 4-3　雙向選擇結構流程圖

# 4-2 雙向選擇結構

表 4-2 雙向選擇結構

| 雙向選擇程式語法 | 程式範例（滿 2000 打九折） |
|---|---|
| if 條件判斷 :<br>　條件成立的敘述<br>else:<br>　條件不成立的敘述 | if cost >= 2000:<br>　print(cost*0.9)<br>else:<br>　print(cost) |
| 說明 | |
| 若 cost 大於等於 2000，則顯示爲 cost 的值打九折，否則顯示 cost 的值 | |

# 範例4-2-1 滿2000 打九折 (ch4\4-2-1- 滿2000 打九折.py)

- 採買物品時，有時會遇到店家為了刺激消費，會使用滿額折扣，如：滿2000 打九折，未滿2000 則不打折，請寫一個程式幫助店家計算顧客所需付出的金額。

- 解題想法
  - 可以使用雙向選擇結構撰寫程式，判斷購買金額是否在2000 元以上，若購買金額在2000 元以上，輸出購買金額乘以0.9；否則依照原價輸出。

# 範例4-2-1 滿2000 打九折 流程圖

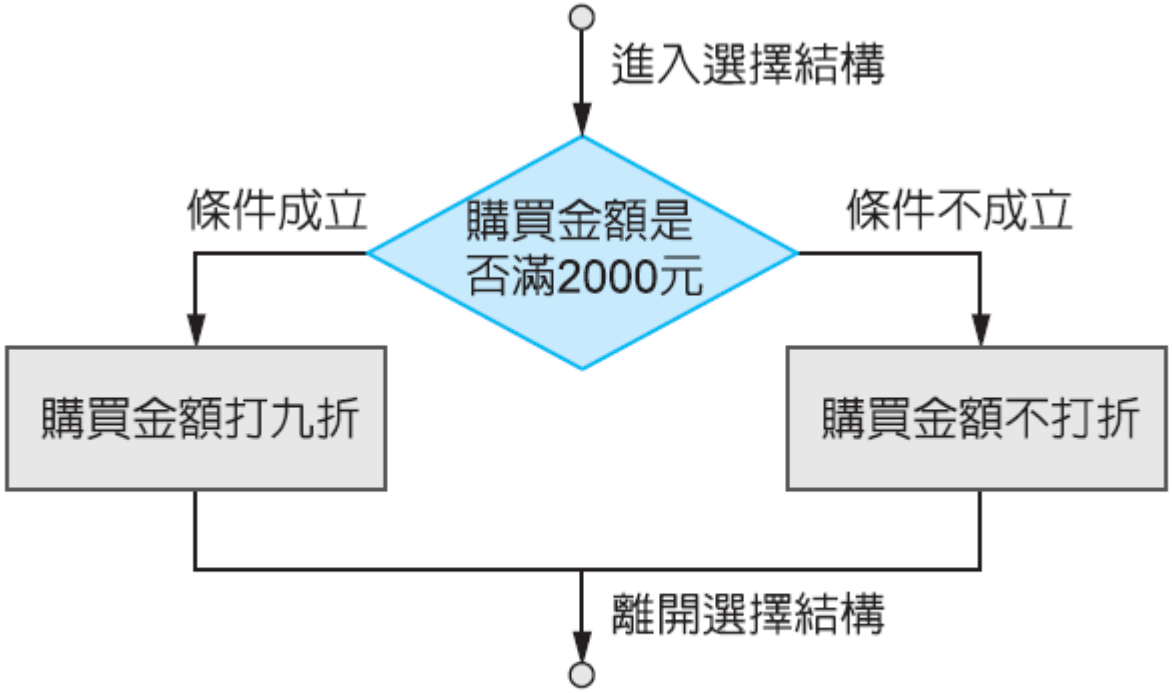



圖 4-4　滿 2000 打九折流程圖

# 範例4-2-1 滿2000 打九折 (ch4\4-2-1- 滿2000 打九折.py)

- 程式碼

```
01 cost = int(input(' 請輸入購買金額？ '))
02 if cost >= 2000:
03     print(cost * 0.9)
04 else:
05     print(cost)
```

## 執行結果

輸入購買金額「2000」，結果顯示在螢幕。

```
請輸入購買金額？ 2000
1800.0
```

# 範例4-2-1 滿2000 打九折 (ch4\4-2-1- 滿2000 打九折.py)

- 程式解說
  - **第1 行**：於螢幕顯示「請輸入購買金額？」，允許使用者輸入購買金額，並經由int 函式轉換成整數物件，變數cost 參考到此整數物件。
  - **第2 到5 行**：條件判斷(if) 對cost 做判斷，大於等於2000 就將該數值打九折( 第2到3 行)，否則該數值不打折( 第4 到5 行)。

# 範例4-2-2 判斷奇偶數 ( ch4\4-2-2- 判斷奇偶數.py)

- 請寫一個程式判斷輸入的值是奇數還是偶數，通常會以求除以2 的餘數，若餘數為0表示輸入的數為偶數，否則輸入的數為奇數。

- 解題想法
  - 可以使用雙向選擇結構撰寫程式，判斷輸入值除以2 的餘數，若餘數不為0，則輸出該數為奇數；否則輸出該數為偶數。

# 範例4-2-2 判斷奇偶數 流程圖

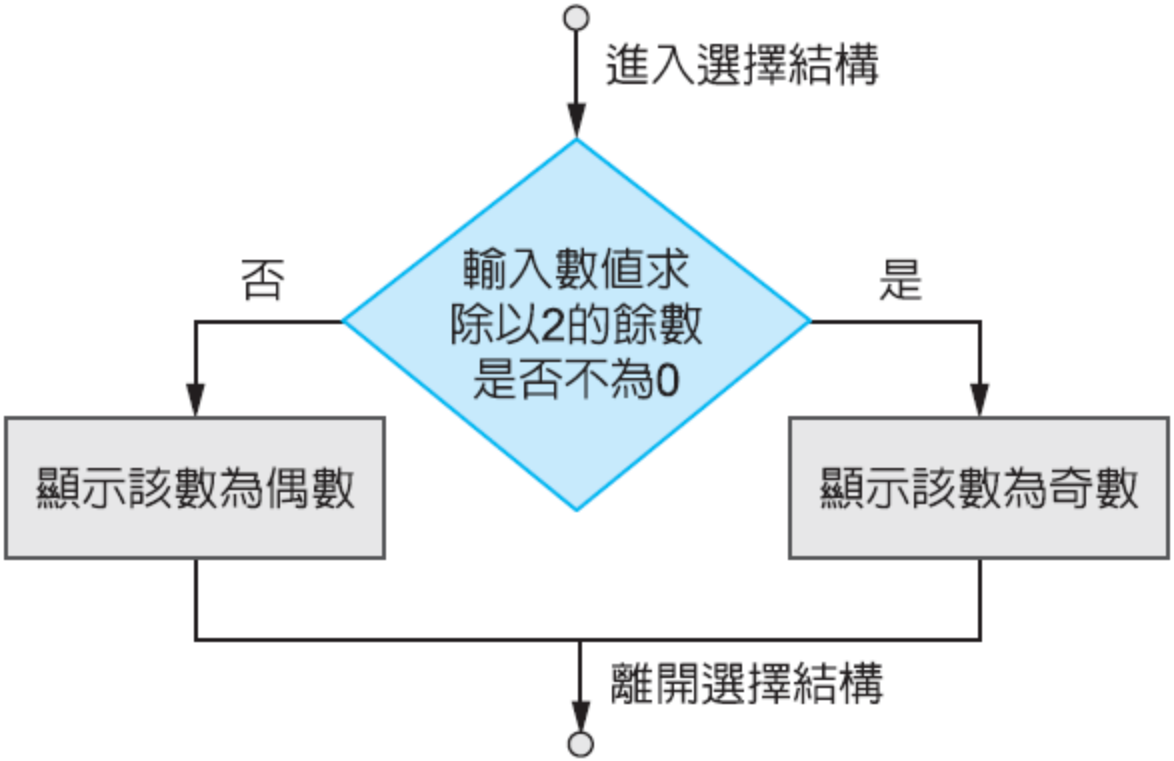



圖 4-5　判斷奇偶數流程圖

# 範例4-2-2 判斷奇偶數 ( ch4\4-2-2- 判斷奇偶數.py)

- 程式碼

```
01  num = int(input(' 請輸入一個整數？ '))
02  if num%2:
03      print(num, ' 為奇數')
04  else:
05      print(num, ' 為偶數')
```

## 執行結果

輸入一個數字「13」，顯示結果在螢幕上。

```
請輸入一個整數？ 13
13 爲奇數
```

# 範例4-2-2 判斷奇偶數 ( ch4\4-2-2- 判斷奇偶數.py)

- 程式解說
  - 第1 行：於螢幕顯示「請輸入一個整數？」，允許使用者輸入整數，並經由int函式轉換成整數物件，變數num 參考到此整數物件。
  - 第2 到5 行：利用條件判斷(if) 對num 做判斷，取2 的餘數，若餘數不為0 則顯示變數num 的值與「為奇數」( 第2 到3 行)，否則餘數為0，則變數num 的值與「為偶數」( 第4 到5 行)。

# 範例4-2-3 三角形判斷 (ch4\4-2-3- 三角形判斷.py)

- 設計一個程式允許輸入三角形三邊長，分別為a、b 與c，根據三角形中任兩邊相加要大於第三邊，判斷是否為三角形。

- 解題想法
  - 可以使用雙向選擇結構撰寫程式，判斷任兩邊相加是否大於第三邊，若任兩邊相加大於第三邊，則顯示「可構成三角形」
  - 否則顯示「無法構成三角形」。任兩邊相加是否大於第三邊，可以結合關係運算子的大於運算子(>) 與邏輯運算子的且運算子(and)完成任兩邊相加是否大於第三邊的判斷。

# 範例4-2-3 三角形判斷 流程圖

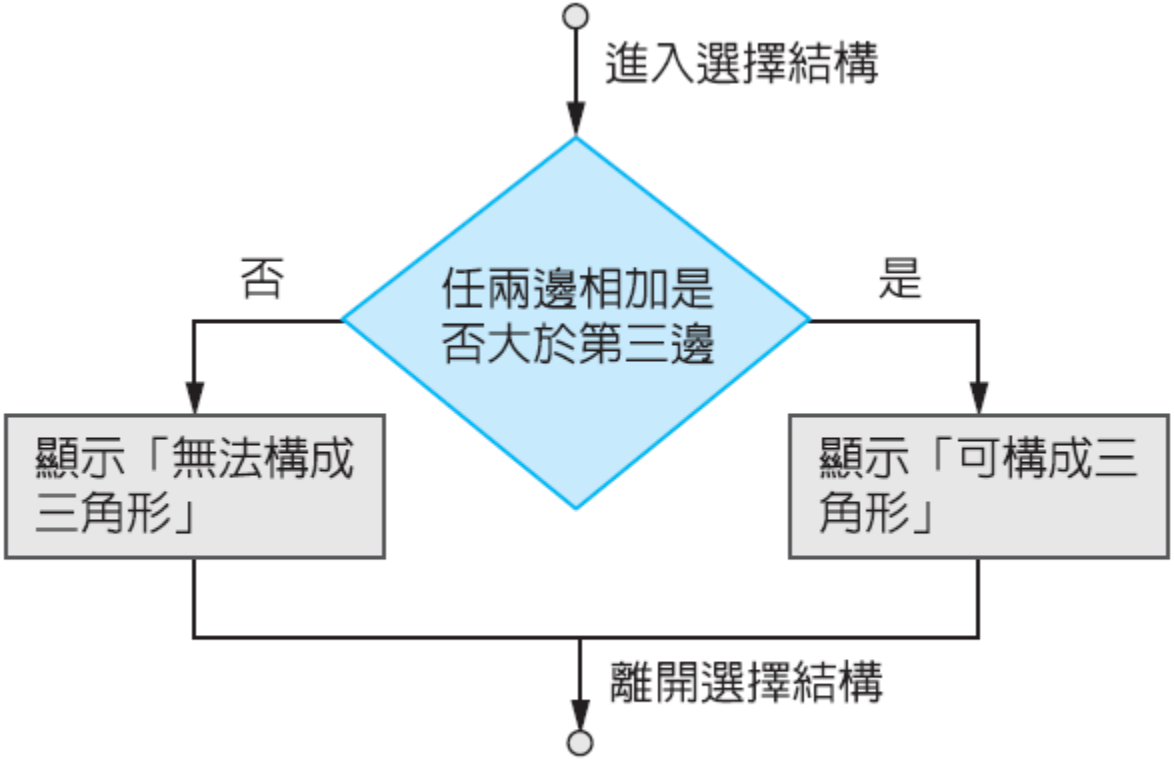



圖 4-6　三角形判斷流程圖

# 範例4-2-3 三角形判斷 (ch4\4-2-3- 三角形判斷.py)

- 程式碼

```
01  a = int(input(' 請輸入三角形邊長a 長度為？ '))
02  b = int(input(' 請輸入三角形邊長b 長度為？ '))
03  c = int(input(' 請輸入三角形邊長c 長度為？ '))
04  if (a<b+c)and(b<a+c)and(c<a+b):
05      print(' 可構成三角形')
06  else:
07      print(' 無法構成三角形')
```

## 執行結果

輸入三邊長分別爲 3，4 與 5，結果顯示在螢幕。

```
請輸入三角形邊長 a 長度爲？ 3
請輸入三角形邊長 b 長度爲？ 4
請輸入三角形邊長 c 長度爲？ 5
可構成三角形
```

# 範例4-2-3 三角形判斷 (ch4\4-2-3- 三角形判斷.py)

- 程式解說
  - 第1 行：於螢幕顯示「請輸入三角形邊長a 長度為？」，允許使用者這輸入整數，並經由int 函式轉換成整數物件，變數a 參考到此整數物件。
  - 第2 行：於螢幕顯示「請輸入三角形邊長b 長度為？」，允許使用者這輸入整數，並經由int 函式轉換成整數物件，變數b參考到此整數物件。

- 第3 行：於螢幕顯示「請輸入三角形邊長c 長度為？」，允許使用者輸入整數，並經由int 函式轉換成整數物件，變數c 參考到此整數物件。
- 第4 到5 行：判斷是否(a<b+c) 且(b<a+c) 且(c<a+b) 條件成立，若條件成立，則顯示「可構成三角形」。
- 第6 到7 行：否則顯示「無法構成三角形」。

# 4-3 多向選擇結構

- 除了單向選擇與雙向選擇外，更廣義的選擇結構是多向選擇
- 意即選擇結構中還可以加入選擇結構，單向選擇與雙向選擇為多向選擇結構的特例，多向選擇結構讓程式有無限可能執行的路徑與狀態。

- 我們可以使用多個if-elif-else 來達成多向選擇結構，以成績與評語對應關係為例來介紹多向選擇結構

- 例如：
  - 假設成績與評語有對應關係，若成績大於等於80 分，評語為「非常好」
  - 否則若成績大於等於60 分，也就是小於80 分且大於等於60 分，評語為「不錯喔」
  - 否則評語為「要加油」，也就是小於60 分，這就是多向選擇結構。

# 選擇結構：多向選擇結構流程圖

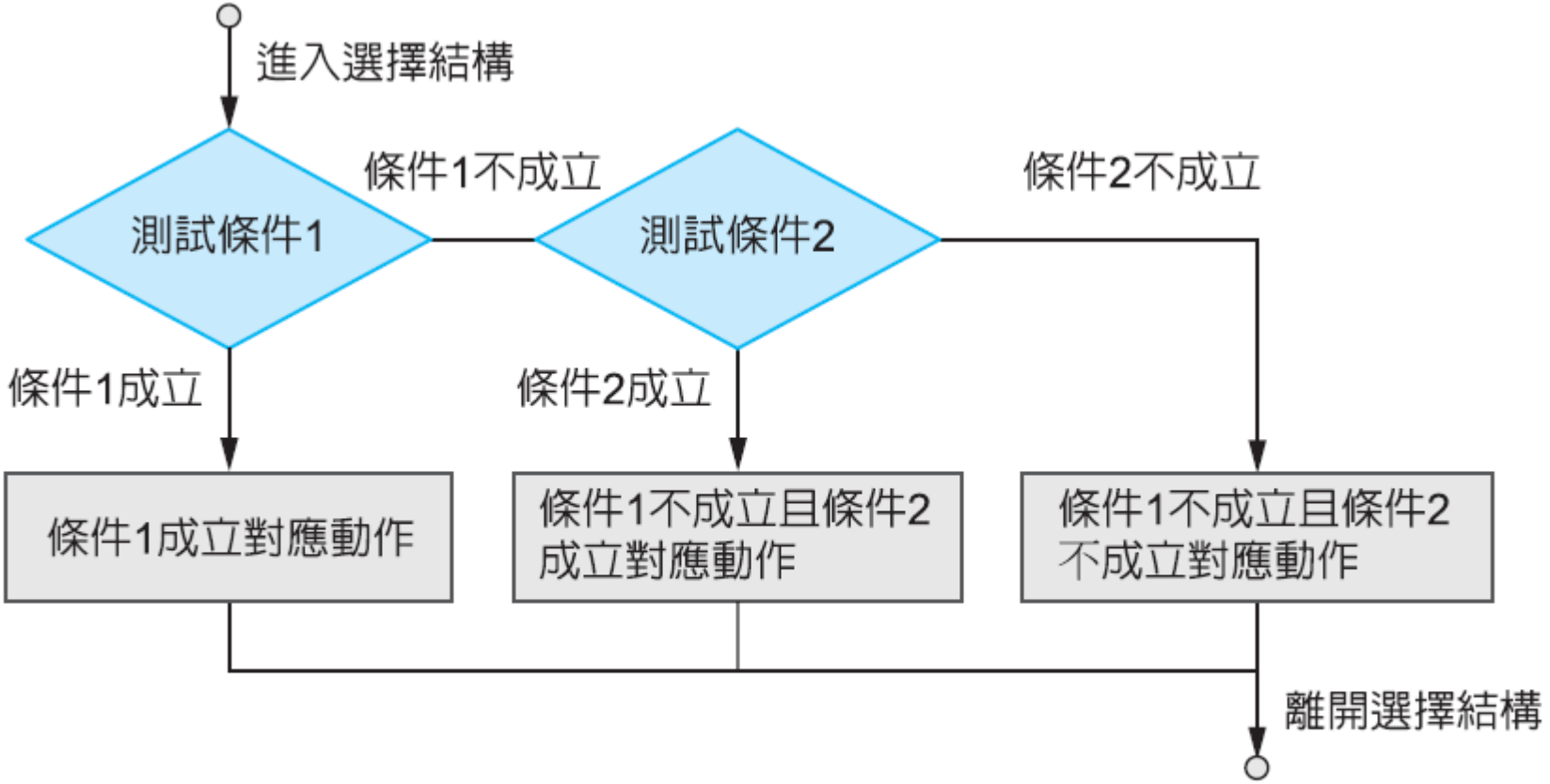



圖 4-7　多向選擇結構流程圖

# 4-3 多向選擇結構

表 4-3 多向選擇結構

| 多向選擇結構語法 | 程式範例（分數與評語） |
| --- | --- |
| if 條件判斷 1:<br>　條件判斷 1 成立的敘述<br>elif 條件判斷 2:<br>　條件判斷 2 成立的敘述<br>else:<br>　條件判斷 2 不成立的敘述 | if score >= 80:<br>　print(' 非常好 ')<br>elif score >= 60:<br>　print(' 不錯喔 ')<br>else:<br>　print(' 要加油 ') |

# 範例4-3-1 分數與評語 (ch4\4-3-1- 分數與評語.py)

- 寫一個程式若成績大於等於80 分，評語為「非常好」，否則若成績大於等於60 分，評語為「不錯喔」，否則評語為「要加油」，將以上敘述表示為表格，如下。

表 4-4　分數與評語

| 成績 | 評語 |
|---|---|
| 成績 >=80 | 非常好 |
| 80> 成績 >=60 | 不錯喔 |
| 成績 <60 | 要加油 |

# 範例4-3-1 分數與評語 (ch4\4-3-1- 分數與評語.py)

- 解題想法
  - 可以使用多向選擇結構撰寫程式，若成績是否大於等於80，則顯示「非常好」，否則若成績大於等於60，則顯示「不錯喔」，否則顯示「要加油」。

# 範例4-3-1 分數與評語 流程圖

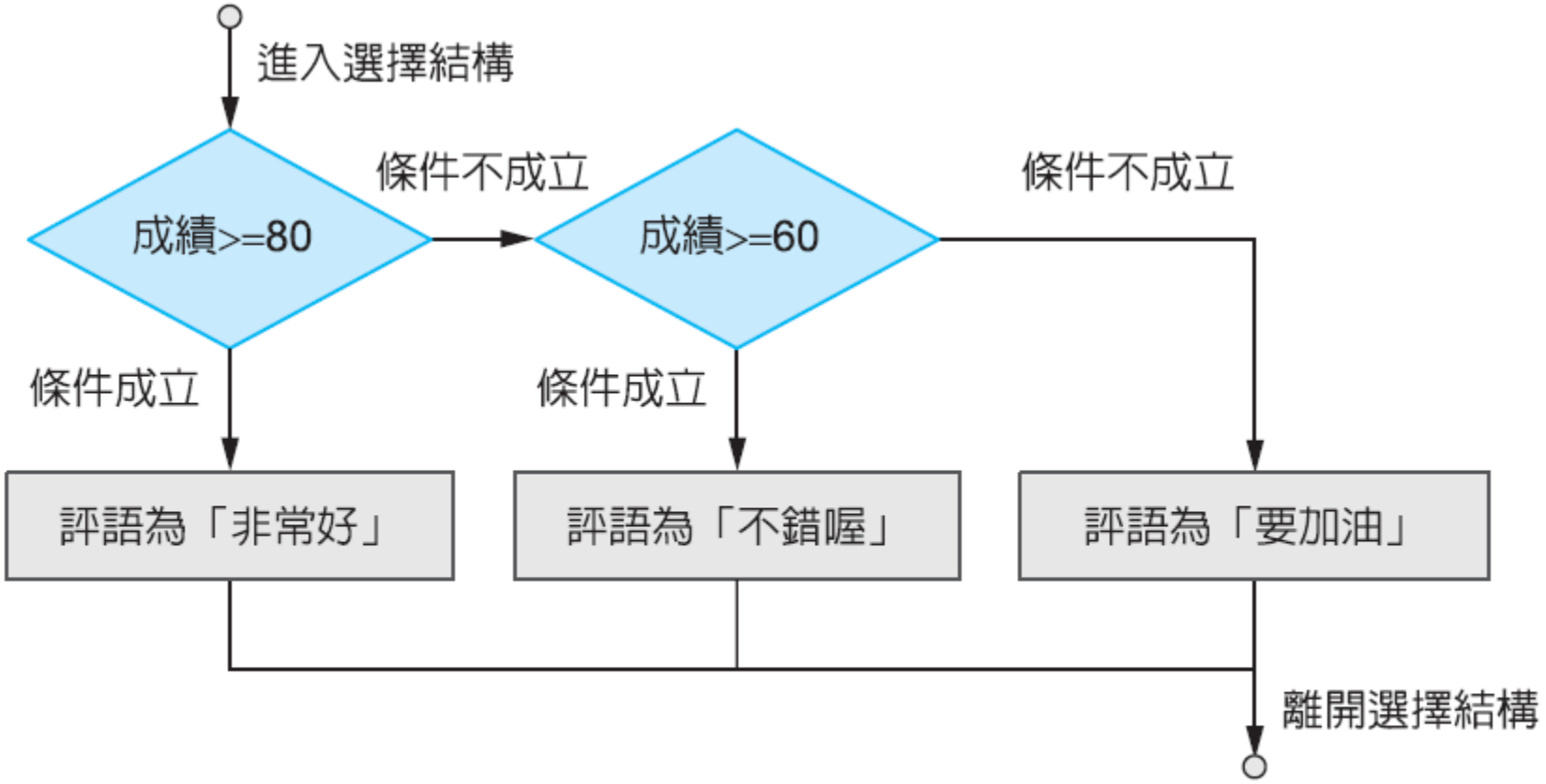



圖 4-8　分數與評語流程圖

# 範例4-3-1 分數與評語 (ch4\4-3-1- 分數與評語.py)

- 程式

```
01  score = int(input(' 請輸入一個成績？ '))
02  if score >= 80:
03      print('非常好')
04  elif score >= 60:
05      print(' 不錯喔')
06  else:
07      print(' 要加油')
```

執行結果

輸入成績「60」，結果顯示在螢幕。

```
請輸入一個成績？ 60
不錯喔
```

# 範例4-3-1 分數與評語 (ch4\4-3-1- 分數與評語.py)

- 程式解說
  - 第1 行：於螢幕顯示「請輸入一個成績？」，允許使用者輸入成績，並經由int 函式轉換成整數物件，變數score 參考到此整數物件。
  - 第2 到7 行：條件判斷(if) 對score 做判斷，若大於等於80 分就輸出「非常好」( 第2 到3 行)，否則若大於等於60 分就輸出「不錯喔」( 第4 到5 行)，否則輸出「要加油」( 第6 到7 行)。

# 範例4-3-2 郵資計算 (ch4\4-3-2- 郵資計算.py)

- 某快遞公司以重量為計算郵資的依據，重量與郵資計算如下表，請寫一個程式協助快
- 遞人員計算郵資，快遞人員只要輸入重量，程式自動計算郵資。

表 4-5　郵資計算

| 重量 | X<=5 公斤 | X>5 且 X<=10 公斤 | X>10 且 X<=15 公斤 | X>15 且 X<=20 公斤 | X>20 公斤 |
|---|---|---|---|---|---|
| 價格 | 50 | 70 | 90 | 110 | 超過 20 公斤無法寄送 |

# 範例4-3-2 郵資計算 (ch4\4-3-2- 郵資計算.py)

- 解題想法
  - 可以使用多向選擇結構撰寫程式，若重量小於等於5 公斤，則顯示「50」，否則若重量小於等於10 公斤，則顯示「70」，若重量小於等於15 公斤，則顯示「90」，若重量小於等於20 公斤，則顯示「110」，否則顯示「超過20公斤無法寄送」。

# 範例4-3-2 郵資計算 流程圖

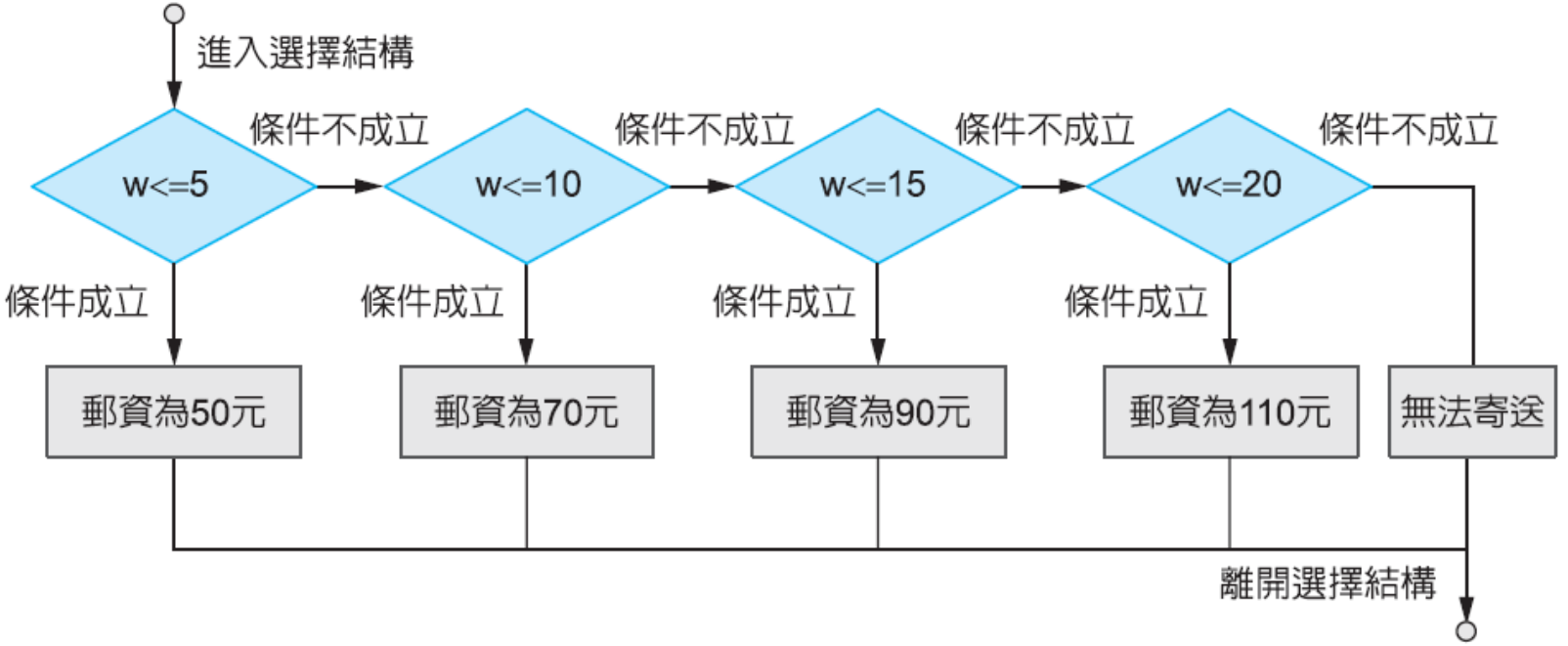



圖 4-9　郵資計算流程圖

# 範例4-3-2 郵資計算 (ch4\4-3-2- 郵資計算.py)

```
01  w = float(input(' 請輸入物品重量？ '))
02  if w <= 5:
03      print(' 所需郵資為50 元')
04  elif w <= 10:
05      print(' 所需郵資為70 元')
06  elif w <= 15:
07      print(' 所需郵資為90 元')
08  elif w <= 20:
09      print(' 所需郵資為110 元')
10  else:
11      print(' 超過20 公斤無法寄送')
```

## 執行結果

請輸入重量「10」，螢幕輸出所需郵資。

```
請輸入物品重量？ 10
所需郵資爲 70 元
```

# 範例4-3-2 郵資計算 (ch4\4-3-2- 郵資計算.py)

- 程式解說
  - 第1 行：於螢幕顯示「請輸入物品重量？」，允許使用者物品重量，並經由float 函式轉換成浮點數物件，變數w 參考到此浮點數物件。
  - 第2 到3 行：判斷w 是否小於等於5，若是則顯示「所需郵資為50 元」。
  - 第4 到5 行：否則若w 是否小於等於10( 隱含重量大於5)，若是則顯示「所需郵資為70 元」。

- 第6 到7 行：否則若w 是否小於等於15( 隱含重量大於10)，若是則顯示「所需郵資為90 元」。
- 第8 到9 行：否則若w 是否小於等於20( 隱含重量大於15)，若是則顯示「所需郵資為110 元」。
- 第10 行到11 行：否則( 隱含重量大於20) 顯示「超過20 公斤無法寄送」。

# 範例4-3-3 BMI 計算 (ch4\4-3-3-BMI 計算.py)

- BMI 常用來判斷肥胖程度，BMI 等於體重（KG）除以身高（M）的平方，會有一個分類標準，假設BMI 與肥胖分級如下。請寫一個程式讓使用者輸入身高與體重，顯示BMI 值與肥胖程度。

表 4-7　BMI 計算

| BMI 值 | 肥胖分級 |
|---|---|
| BMI ＜ 18 | 體重過輕 |
| 18 ≦ BMI ＜ 24 | 體重正常 |
| 24 ≦ BMI ＜ 27 | 體重過重 |
| 27 ≦ BMI | 體重肥胖 |

# 範例4-3-3 BMI 計算 (ch4\4-3-3-BMI 計算.py)

- 解題想法
  - 可以使用多向選擇結構撰寫程式，若BMI 值小於18，則顯示「體重過輕」，否則若BMI 值小於24，則顯示「體重正常」，若BMI 值小於27，則顯示「體重過重」，否則顯示「體重肥胖」。

# 範例4-3-3BMI 計算 流程圖

流程圖

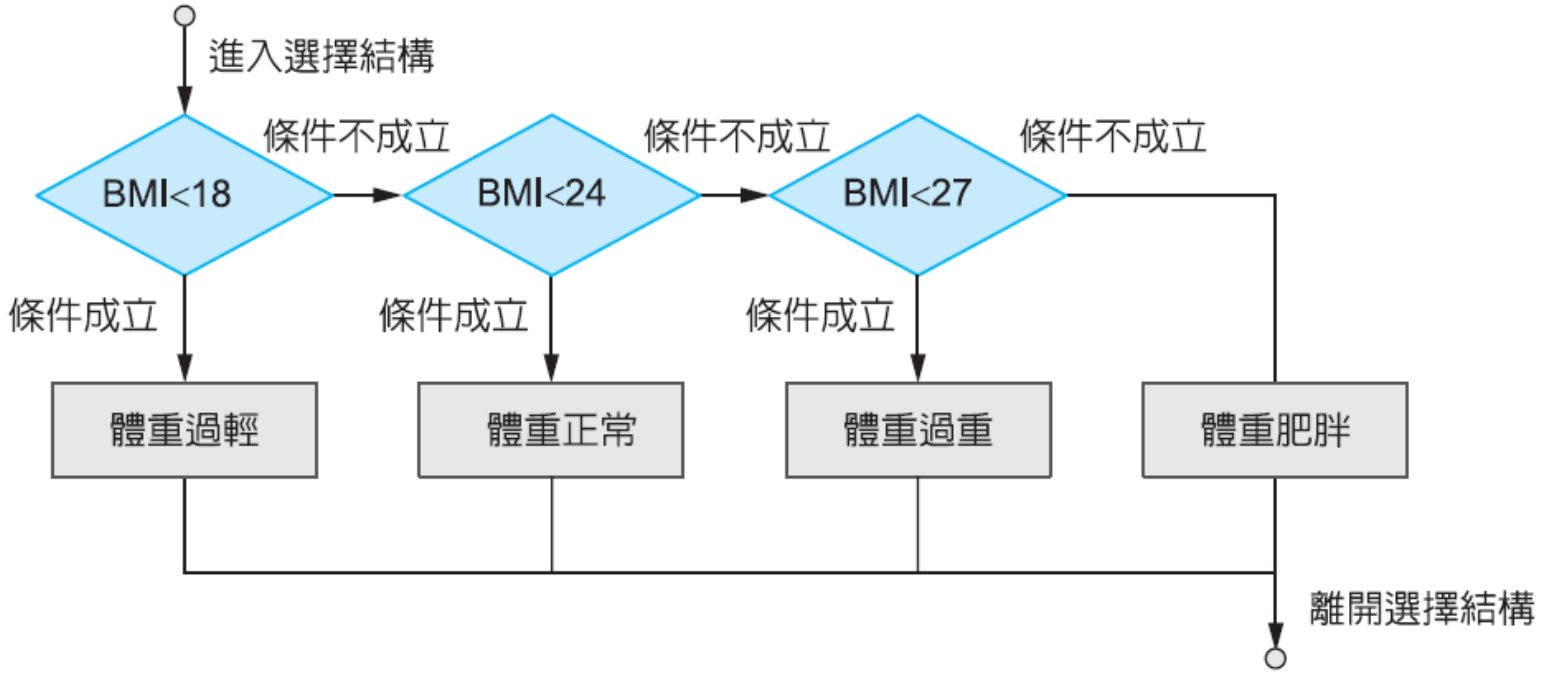



圖 4-10　BMI 計算流程圖

# 範例4-3-3 BMI 計算 (ch4\4-3-3-BMI 計算.py)

```
01  w = float(input(' 請輸入體重(KG) ？ '))
02  h = float(input(' 請輸入身高(M) ？ '))
03  bmi = w/(h*h)
04  print('BMI 為', bmi)
05  if bmi < 18:
06      print(' 體重過輕')
07  elif bmi < 24:
08      print(' 體重正常')
09  elif bmi < 27:
10      print(' 體重過重')
11  else:
12      print(' 體重肥胖')
```

# 範例4-3-3 BMI 計算 (ch4\4-3-3-BMI 計算.py)

## 執行結果

按請輸入體重「75」，身高「1.7」，螢幕輸出 BMI 值與肥胖分級。

```
請輸入體重 (KG) ？ 75
請輸入身高 (M) ？ 1.7
BMI 爲 25.95155709342561
體重過重
```

# 範例4-3-3 BMI 計算 (ch4\4-3-3-BMI 計算.py)

- 程式解說
  - 第1 行：於螢幕顯示「請輸入體重(KG) ？」，允許使用者輸入體重，並經由float函式轉換成浮點數物件，變數w 參考到此浮點數物件。
  - 第2 行：於螢幕顯示「請輸入身高(M) ？」，允許使用者輸入身高，並經由float 函式轉換成浮點數物件，變數h 參考到此浮點數物件。
  - 第3 行：計算所得BMI 值，指定給變數bmi。

- 第4 行：輸出BMI 值於螢幕。
- 第5 到6 行：判斷所計算出的BMI 值是否小於18，若是則顯示「體重過輕」。
- 第7 到8 行：否則判斷所計算出的BMI 值是否小於24( 隱含BMI 值大於等於18)，若是則顯示「體重正常」。

- 第9 到10 行：否則判斷所計算出的BMI 值是否小於27( 隱含BMI 值大於等於24)，若是則顯示「體重過重」。
- 第11 到12 行：否則( 隱含BMI 值大於等於27) 顯示「體重肥胖」。

# 4-4　條件判斷與運算子「in」(ch4\ 條件判斷與運算子「in」.py)

- 可以使用條件判斷與「in」測試資料容器 tuple、串列、字典與集合是否包含某個元素。

# 4-4-1 判斷tuple 是否包含某個元素

使用條件判斷與「in」進行判斷，程式如下。

| 程式 | 執行結果 |
| --- | --- |
| a = (1,2,3,4)<br>if 1 in a:<br>　print(' 數字 1 在 tuple a 中 ')<br>else:<br>　print(' 數字 1 不在 tuple a 中 ') | 數字 1 在 tuple a 中 |

# 4-4-2 判斷串列是否包含某個元素

使用條件判斷與「in」進行判斷，程式如下。

| 程式 | 執行結果 |
| --- | --- |
| a = list('abcdefghijklmnopqrstuvwxyz')<br>if 'q' in a:<br>  print('q 在串列 a 中 ')<br>else:<br>  print('q 不在串列 a 中 ') | q 在串列 a 中 |

# 4-4-3 判斷字典是否包含某個元素

使用條件判斷與「in」進行判斷，程式如下。

| 程式 | 執行結果 |
| --- | --- |
| lang1={'早安':'Good Morning','謝謝':'Thank You'}<br>if '謝謝' in lang1:<br>  print('謝謝的英文爲', lang1['謝謝'])<br>else:<br>  print('查不到謝謝的英文') | 謝謝的英文爲 Thank You |

# 4-4-4　判斷集合是否包含某個元素

使用條件判斷與「in」進行判斷，程式如下。

| 程式 | 執行結果 |
| --- | --- |
| a = set('tiger')<br>if 't' in a:<br>　print('t 在集合 a 內 ')<br>else:<br>　print('t 不在集合 a 內 ') | t 在集合 a 內 |